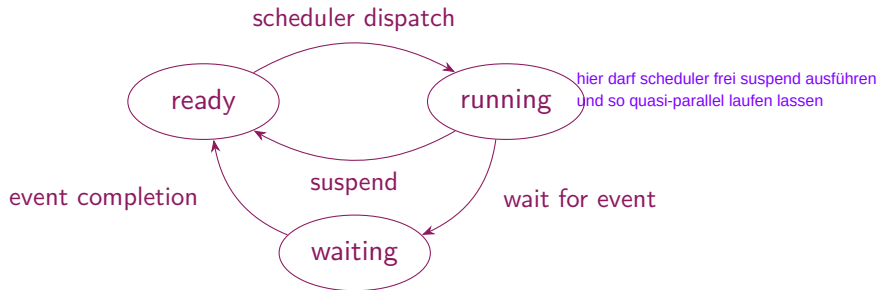
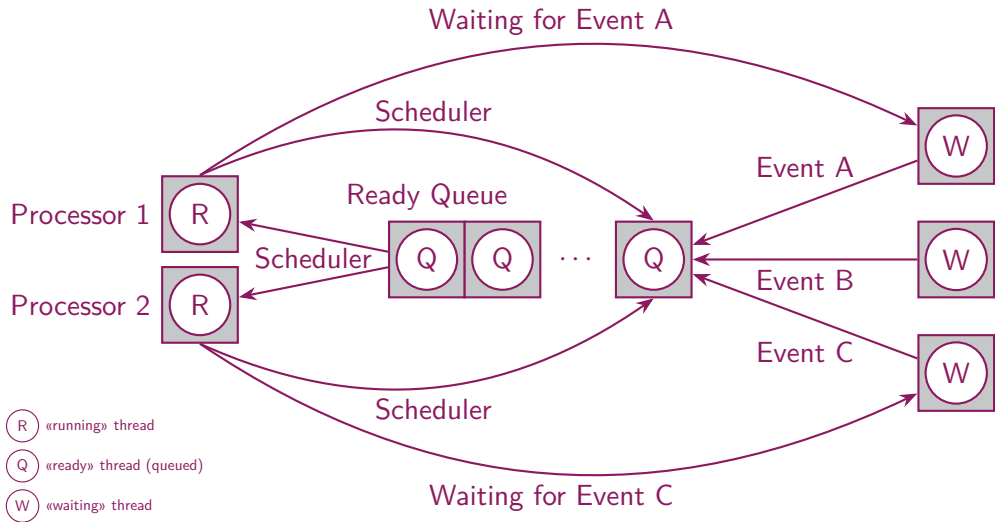


Zustände eines Threads

Grundmodell



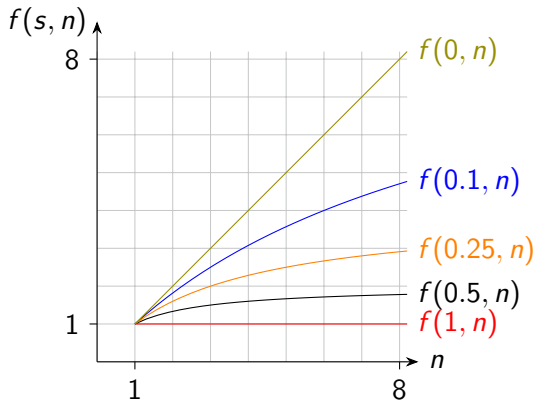
Ready-Queue



Amdahls Regel

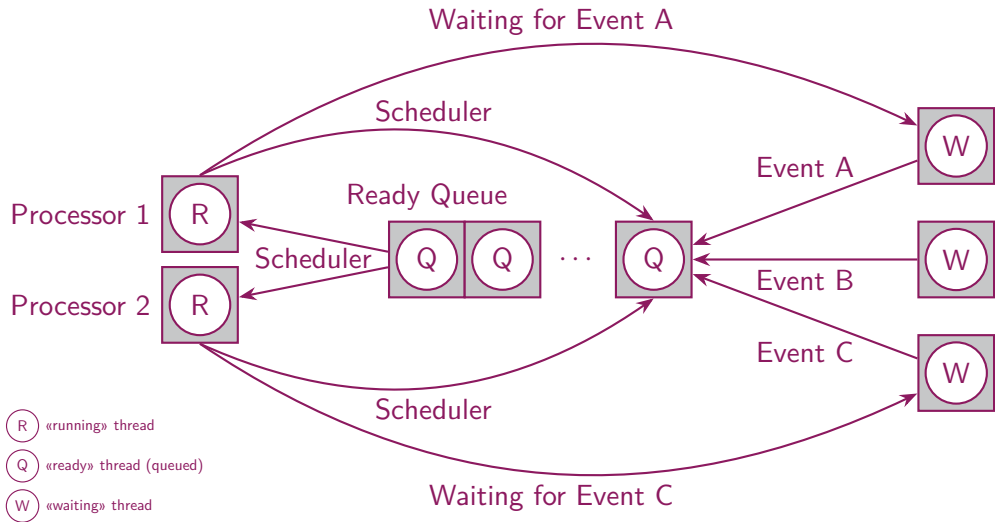
Bedeutung

kein Programm ist 100% parallelisiert
wegen z.B. IO
gemäß DSy auch Zugriff auf DB



- Abschätzung einer oberen Schranke für den maximalen Geschwindigkeitsgewinn
- Nur wenn alles parallelisierbar ist, ist der Speedup proportional und maximal $f(0, n) = n$
- Sonst ist der Speedup mit höherer Prozessor-Anzahl immer geringer

Ready-Queue

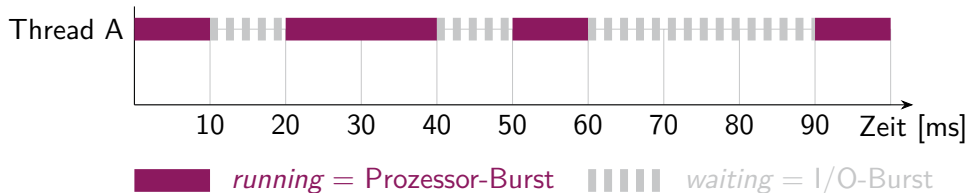


Bursts

Prozessor-Burst Intervall, in dem ein Thread den Prozessor in einem parallelen System voll belegt, also vom Eintritt in *running* bis zum nächsten *waiting*

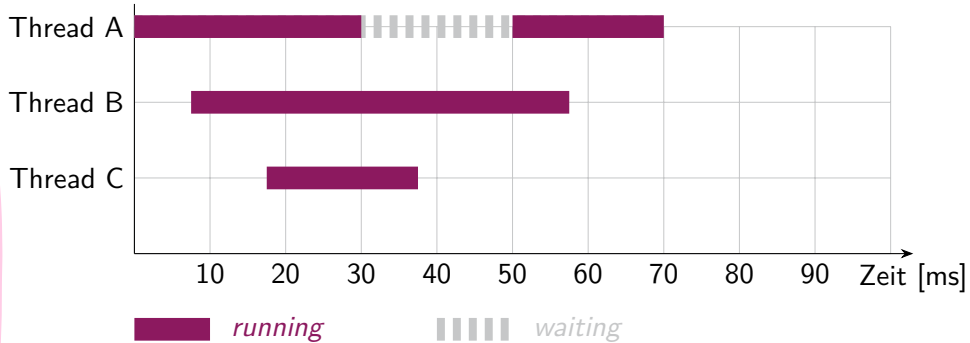
I/O-Burst Intervall, in dem ein Thread den Prozessor nicht benötigt, also vom Eintritt in *waiting* bis zum nächsten *running*

Jeder Thread kann als Abfolge von Prozessor-Bursts und I/O-Bursts betrachtet werden.



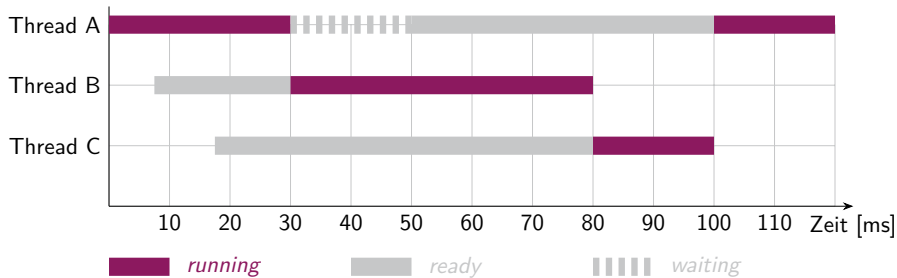
Parallele Ausführung

Beispiel mit 3 Threads auf 3 Prozessoren



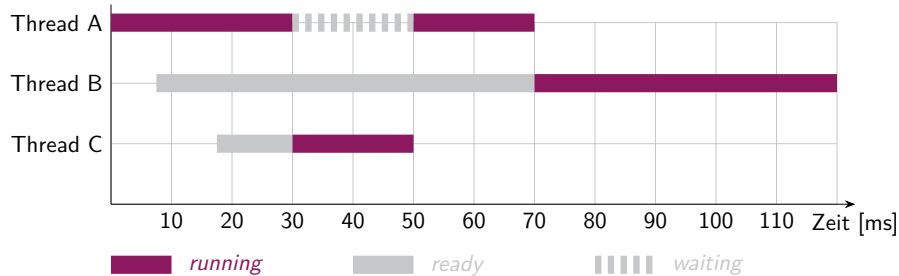
FCFS-Strategie (First Come, First Served)

Beispiel mit 3 Threads auf 1 Prozessor



SJF-Strategie (Shortest Job First)

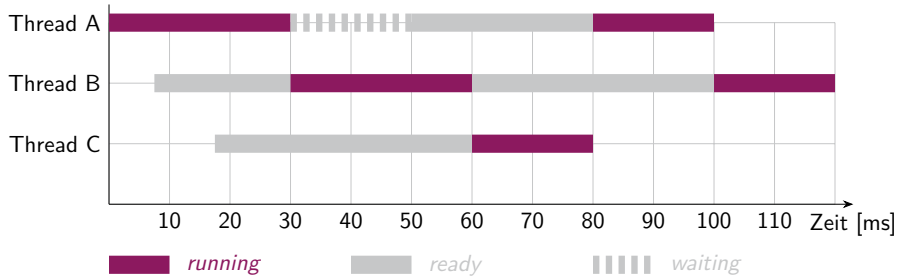
Beispiel mit 3 Threads auf 1 Prozessor



weniger graue balken, weniger waiting time

Round-Robin-Scheduling

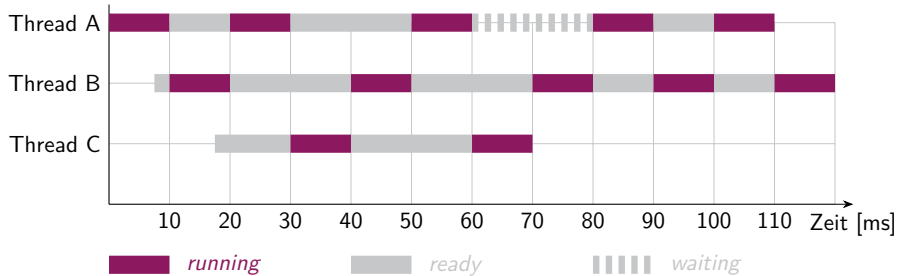
Beispiel mit 3 Threads auf 1 Prozessor, Time-Slice 30 ms



sehr lange wartezeit (ready balken)

Round-Robin-Scheduling

Beispiel mit 3 Threads auf 1 Prozessor, Time-Slice 10 ms



FCFS round robin

ASCII – American Standard Code for Information Interchange

128 definierte Zeichen $\Rightarrow$ 7 Bit, 00 bis 7F

33 Kontrollzeichen, viele davon obsolet

10 Ziffern:

0 - 9

33 Interpunktionszeichen:

! " # ...

26 Grossbuchstaben:

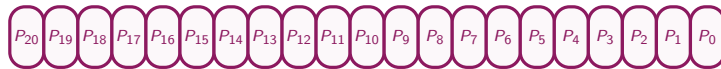
A - Z

26 Kleinbuchstaben:

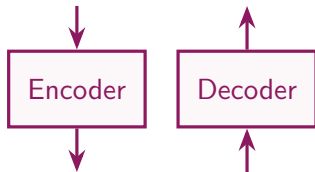
a - z

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH 1	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SB	ESC	FS	GS	RS	US
2	␣	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Verschiedene Encodings – Definitionen für folgende Folien



Codepoint

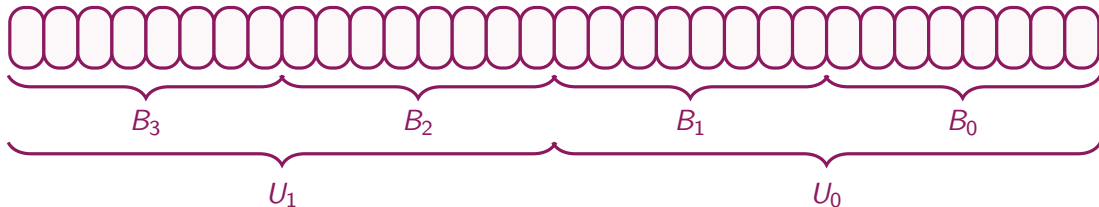


P_i : i -tes Bit des unkodierten CPs

U_i : i -te Code-Unit des kodierten CPs

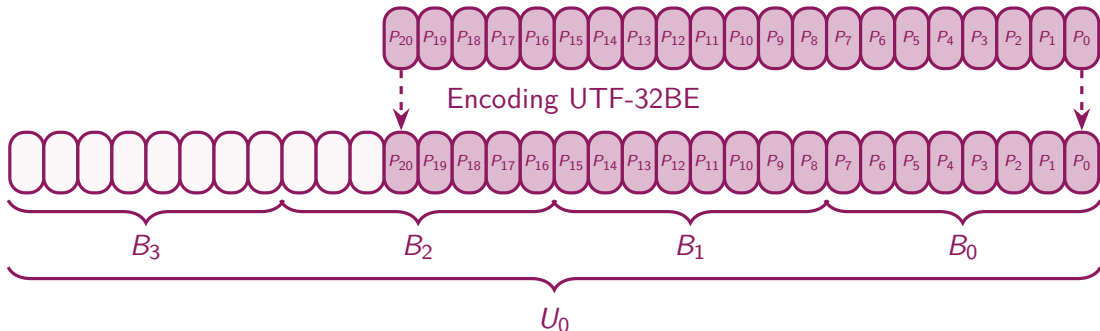
B_i : i -tes Byte des kodierten CPs

Encoding (Beispiel):



UTF-32

- 32 Bit pro Code-Unit
 - Jeder Code-Point kann mit einer Code-Unit dargestellt werden: $U_0 = 0 \dots 0 P_{20} \dots P_0$
- Häufig intern in Programmen verwendet
 - Wegen Limitierung auf 21 Bit werden die 11 oberen Bits oft «zweckentfremdet»
- Bei byteweisen Übertragungen muss Endianness beachtet werden
 - Endianness gibt an, mit welchem Ende die Folge anfängt eigentlich verkehrt rum
 - Little-Endian: $U_0 = B_0 B_1 B_2 B_3$
 - Big-Endian: $U_0 = B_3 B_2 B_1 B_0$

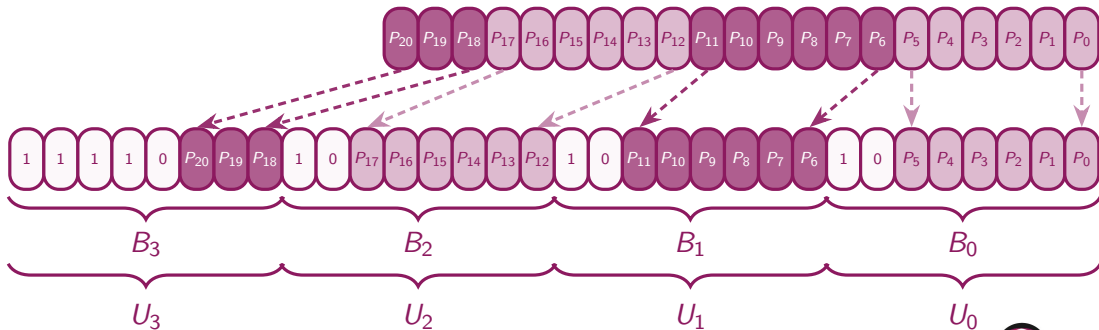


UTF-8 Encoding

4 Bytes

Codepoint in $[1'0000, 10'FFFF]$: Bis zu 21 signifikante Bits

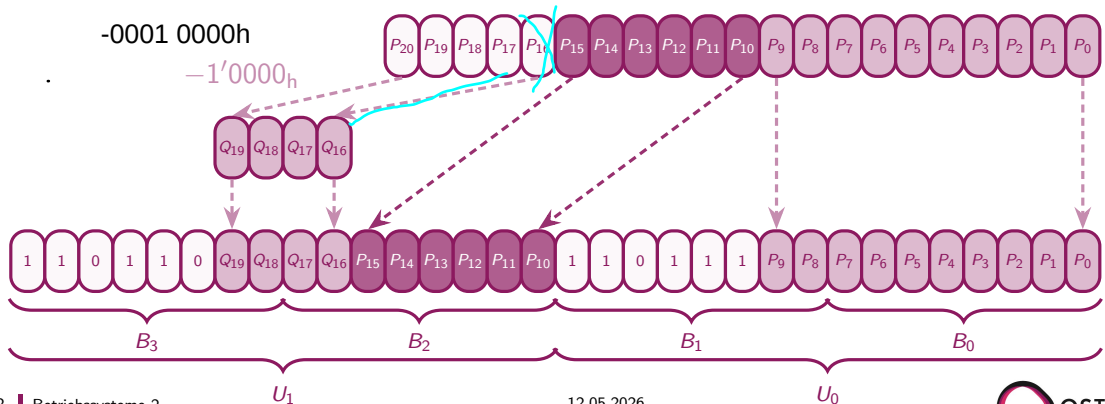
- $U_3 = B_3 = 11110 P_{20} P_{19} P_{18} (+ F0_h)$
- $U_2 = B_2 = 10 P_{17} \dots P_{12} (+ 80_h)$
- $U_1 = B_1 = 10 P_{11} \dots P_6 (+ 80_h)$
- $U_0 = B_0 = 10 P_5 \dots P_0 (+ 80_h)$



UTF-16 Encoding, 4 Bytes

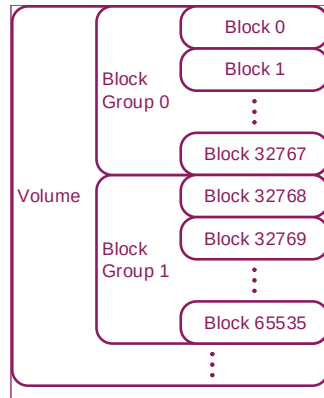
CP in $[1'0000, 10'FFFF]$

- $Q = P - 1'0000_h$ (also ist Q in $[0, F'FFFF]$ und $Q_{15} \dots Q_0 = P_{15} \dots P_0$)
- $U_1 = 1101'10 Q_{19} \dots Q_{10} (+ D800_h)$; $U_0 = 1101'11 Q_9 \dots Q_0 (+ DC00_h)$
- Die beiden CUs heissen Surrogate-Pairs



Blockgruppe

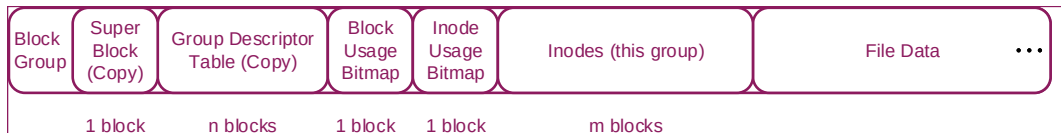
- Volume wird in Blockgruppen unterteilt
- Blockgruppe:
 - Mehrere aufeinanderfolgende Blöcke: bis zu 8 mal der Anzahl Bytes in einem Block
Beispiel: Bei Blockgrösse 4 KB sind bis zu 32 K Blöcke in einer Gruppe
 - Blöcke je Gruppe gleich für alle Gruppen



Blockgruppe

Layout:

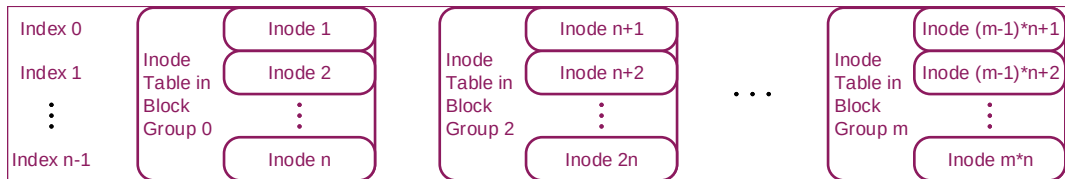
- Block 0: Kopie des Superblocks
- Block 1 bis n : Kopie der Gruppendeskriptorentabelle
- Block $n + 1$: Block-Usage-Bitmap mit einem Bit je Block der Gruppe
- Block $n + 2$: Inode-Usage-Bitmap mit einem Bit je Inode der Gruppe
- Block $n + 3$ bis $n + m + 2$: Tabelle aller Inodes in dieser Gruppe
- Block $n + m + 3$ bis Ende der Gruppe: Blöcke der eigentlichen Daten



Inodes

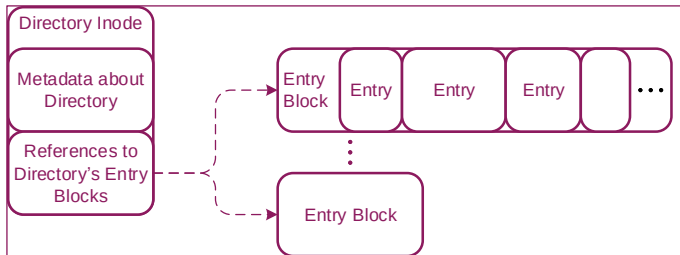
Lokalisierung eines Inodes:

- Alle Inodes aller Blockgruppen gelten als eine grosse Tabelle
- Zählung der Inodes startet mit 1
- $\text{Blockgruppe} = (\text{Inode} - 1) / \text{«Anzahl Inodes pro Gruppe»}$
- $\text{«Index des Inodes in Blockgruppe»} = (\text{Inode} - 1) \% \text{«Anzahl Inodes pro Gruppe»}$
- Sektor und Offset können dann anhand der Daten aus dem Superblock bestimmt werden



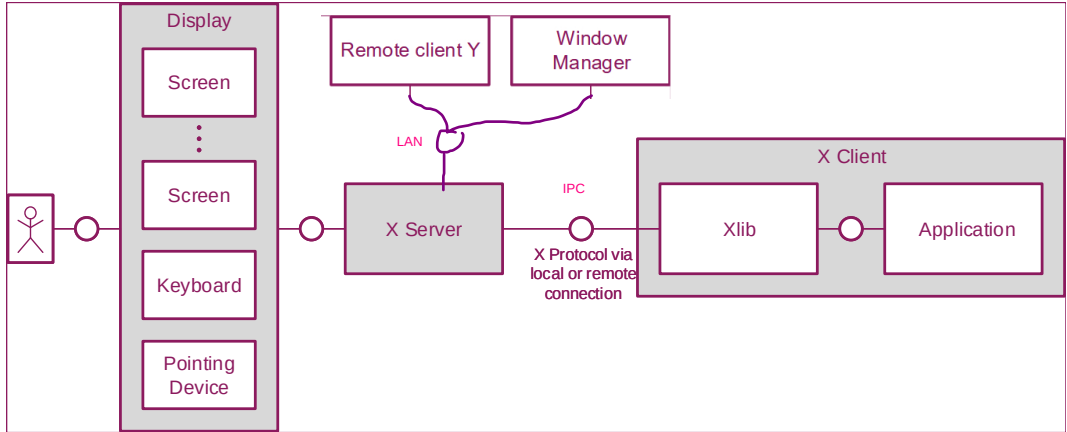
Verzeichnisse

- Verzeichnis enthält Dateieinträge
 - Genauer: Inode, dessen Datenbereich Dateieinträge enthält

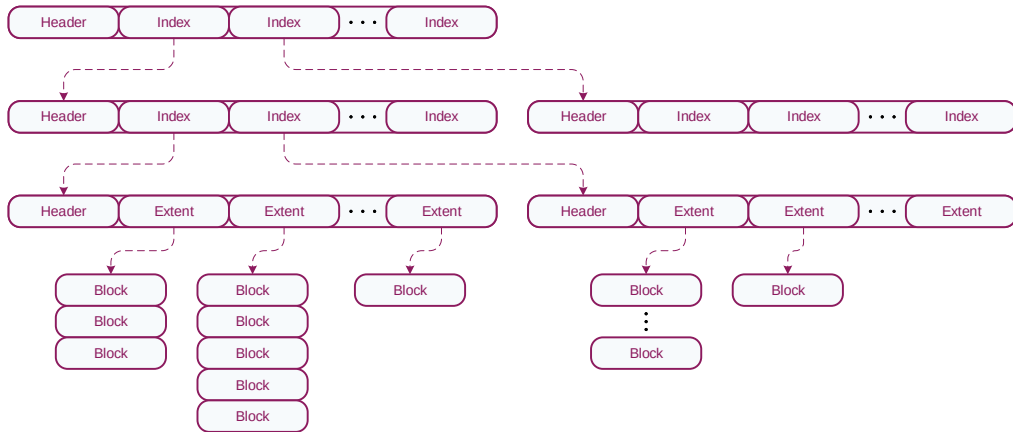


- Zwei automatisch angelegte Einträge
 - «.»: Dateieintrag mit eigenem Inode (Verweis auf sich selbst)
 - «..»: Dateieintrag mit Inode des Elternverzeichnisses (Verweis auf Elternverzeichnis)
- Wurzelverzeichnis ist Inode Nummer 2

Überblick



Extent Tree Beispiel



Extent Trees

Index-Block:

- Enthält einen eigenen Tree-Header — Tiefe ist um 1 kleiner als übergeordneter Knoten
- Enthält Referenzen auf die Kind-Knoten: je nach Tiefe entweder Index-Einträge oder Extents
- `i_block[0...14]` kann als (sehr kleiner) Index-Block aufgefasst werden

